ERRORES Y EXCEPCIONES EN JAVA

Desarrollo de Sistemas Orientado a Objetos

¿QUÉ ES UN ERROR?

Definición y cuándo aparece

Un Error en Java es una condición grave e irrecuperable que ocurre en tiempo de ejecución. Es una subclase de Throwable, pero a diferencia de Exception, generalmente no puede ni debe ser manejada por el código de la aplicación.

Los Errors indican fallas del sistema o de la JVM, no de la lógica del programa. Por eso, capturarlos con catch es una mala práctica: señalan que algo muy grave salió mal.

Ejemplos comunes: OutOfMemoryError (la JVM se quedó sin memoria), StackOverflowError (recursión infinita), VirtualMachineError (fallo interno de la JVM).

JERARQUÍA EN JAVA: Error vs Exception

ERROR (irrecuperable)

- **OutOfMemoryError**
La JVM se quedó sin memoria heap. No hay forma de recuperarse.

- **StackOverflowError**
Recursión infinita o pila de llamadas agotada.

- **VirtualMachineError**
Falla interna grave de la JVM.

EXCEPTION (manejable)

- **IOException / SQLException**
Checked: el compilador obliga a manejarlas con try/catch o throws.

- **RuntimeException**
Unchecked: NullPointerException, IllegalArgumentException, etc. No es obligatorio capturarlas.

- **Excepción personalizada**
Extendé Exception (checked) o RuntimeException (unchecked) según el caso.

La palabra clave throw

Lanzar una excepción significa crear un objeto que describe el problema y enviarlo al sistema con throw. Ejemplo: throw new IllegalArgumentException("La edad no puede ser negativa.");

Con throws en la firma del método se declara que puede lanzar excepciones checked: public void leerArchivo(String ruta) throws IOException { ... }

Excepciones checked: el compilador obliga a manejarlas (IOException, SQLException). Unchecked: subclases de RuntimeException, no es obligatorio capturarlas (NullPointerException, IllegalArgumentException).

Capturar y volver a lanzar

Relanzar implica capturar una excepción y lanzarla nuevamente para que la maneje una capa superior. Puede relanzarse igual (throw e;) o envuelta en otra de más alto nivel.

Ejemplo: catch (SQLException e) { throw new RuntimeException("Error al acceder a la BD.", e); } — se preserva la causa original con el segundo argumento.

¿Cuándo relanzar? Para traducir excepciones técnicas en excepciones de dominio, para enriquecer con contexto adicional, o cuando la capa actual no puede resolver el problema.

try / catch / finally

El bloque try envuelve el código que puede lanzar excepciones. El bloque catch captura y maneja una excepción específica. Se pueden encadenar varios catch, del más específico al más general.

El bloque finally se ejecuta SIEMPRE, haya o no excepción. Es ideal para liberar recursos (cerrar archivos, conexiones). try-with-resources cierra automáticamente objetos AutoCloseable.

Regla de orden: catch (IOException e) antes que catch (Exception e). Capturar Exception como comodín genérico es válido solo como último recurso, nunca dejarlo vacío.

BUENAS PRÁCTICAS

Solo para lo excepcional

No usés excepciones para controlar el flujo normal del programa. Si es previsible, usá if/else.

Mensajes descriptivos

El mensaje debe explicar QUÉ falló y en qué contexto, no solo el tipo de error.

Nunca ignorar excepciones

Un catch vacío oculta errores graves. Como mínimo, loguear la excepción con e.printStackTrace() o un logger.

Liberar recursos siempre

Usá finally o try-with-resources para cerrar archivos, conexiones y streams, incluso si hay excepción.

EXCEPCIONES COMUNES EN JAVA

NullPointerException

Se accede a un método o campo sobre una referencia null. Tipo: Unchecked (Runtime).

ArrayIndexOutOfBoundsException

Se intenta acceder a un índice fuera del rango del arreglo. Tipo: Unchecked (Runtime).

IOException

Error en operaciones de entrada/salida: archivos, streams, redes. Tipo: Checked — obligatorio manejarla.

IllegalArgumentException

Se pasó un argumento inválido a un método. Tipo: Unchecked (Runtime). Muy útil en validaciones.

EXCEPCIÓN PERSONALIZADA

Ejemplo: SaldoInsuficienteException

Para crear una excepción propia, extendé Exception (checked) o RuntimeException (unchecked) y agregá un constructor con mensaje. Ejemplo: public class SaldoInsuficienteException extends Exception { public SaldoInsuficienteException(double monto) { super("Saldo insuficiente para extraer $" + monto); } }

Uso en la clase: public void extraer(double monto) throws SaldoInsuficienteException { if (monto > saldo) throw new SaldoInsuficienteException(monto); saldo -= monto; }

Captura: try { cuenta.extraer(5000); } catch (SaldoInsuficienteException e) { System.out.println(e.getMessage()); } — el mensaje muestra exactamente qué monto generó el problema.

CÓDIGO: SaldoInsuficienteException

```
public class SaldoInsuficienteException
extends Exception {

  private final double monto;
  public SaldoInsuficienteException(double
monto) {
    super("Saldo insuficiente: $" + monto);
    this.monto = monto;
   }

  public double getMonto() {
    return monto;
  }
}
```

```
public void extraer(double monto)
    throws SaldoInsuficienteException {
  if (monto > saldo)
    throw new SaldoInsuficienteException(monto);
  saldo -= monto;
}

// Captura:
try {
  cuenta.extraer(5000);
} catch (SaldoInsuficienteException e) {
  System.out.println(e.getMessage());
}

// Resultado:
"Saldo insuficiente: $5000.0"
```



RESUMEN

throw lanza · catch atrapa · finally siempre se ejecuta
Un buen manejo de excepciones hace el código más robusto, legible y mantenible. Diseñá excepciones que comuniquen problemas reales y tratá solo lo que podés resolver.